

LABORATORY OBSERVATION / RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 6

Student Name: K. Sirisha

Roll No.: A24126552260

Section: B

MIDDLEWARE IN EXPRESS.JS (CO-2)

1. TITLE

What is Middleware in Express.js? Explain How Middleware Works in the Request-Response Cycle and Develop an Example of Middleware Used for Logging Incoming Requests.

2. AIM

To study the concept of middleware in Express.js and its working in the request-response cycle, and to develop an Express application in which a custom middleware logs every incoming request.

3. OBJECTIVE

- To understand what Express.js is and what middleware means.
- To understand the middleware function signature (req, res, next) and the role of next().
- To understand how middleware functions are executed in order in the request-response cycle.
- To differentiate between application-level, router-level, built-in, third-party, and error-handling middleware.
- To develop a custom logging middleware that records the method, URL, status code, and response time of every request.
- To use a route-specific middleware to allow or block a request, and to handle 404 errors.
- To test the application using a browser and an API client (Postman or curl).

4. THEORY

1. What is Express.js?

Express.js is a minimal and flexible web application framework for Node.js. It simplifies the creation of web servers and REST APIs by providing routing, helper methods for the request and response objects, and support for middleware, so that the developer does not have to work directly with the low-level http module.

2. What is Middleware?

Middleware is a function that has access to the request object (req), the response object (res), and the next middleware function in the request-response cycle (next). A middleware function can:

- execute any code, such as logging or checking a condition;
- make changes to req and res, for example by adding a property or parsing the body;
- end the request-response cycle by sending a response; or
- call next() to pass control to the next middleware function.

If a middleware function neither sends a response nor calls next(), the request is left hanging and the client keeps waiting. The general form of a middleware function is:

```
function middleware(req, res, next) {  
  // 1. do some work (log, check, modify req / res)  
  next();           // 2. pass the control to the next function  
}
```

3. Working of Middleware in the Request-Response Cycle

When a client sends an HTTP request, Express passes it through the middleware functions in the same order in which they were registered using app.use() or in the route definition. Each function either ends the cycle by sending a response or calls next(). The route handler is the last stage and it sends the response. The order of registration is therefore important: a logger that is registered first will see every request.

```
Client (browser / Postman / curl)  
  |  
  | HTTP request (for example GET /admin)  
  v  
+-----+  
| 1. requestLogger (application) | stores the time, registers a 'finish' listener,  
+-----+ then calls next()  
  | next()  
  v  
+-----+  
| 2. express.json() (built-in) | parses a JSON body into req.body, calls next()  
+-----+  
  | next()  
  v  
+-----+ key missing  
| 3. checkApiKey (route-specific) | -----> 401 response, cycle ends  
+-----+ (next() is not called)  
  | next() (key is valid)  
  v  
+-----+  
| 4. Route handler | res.send() / res.json()  
+-----+  
  |  
  v  
Response returns to the client -> logger prints "GET /admin -> 200 (3 ms)"
```

4. Types of Middleware

Type	Description	Example
Application-level	Bound to the app object using <code>app.use()</code> or <code>app.get()</code> / <code>app.post()</code> . It can run for every request or only for a particular route.	<code>app.use(requestLogger)</code>
Router-level	Works in the same way but is bound to an instance of <code>express.Router()</code> .	<code>router.use(auth)</code>
Built-in	Provided with Express itself.	<code>express.json()</code> , <code>express.urlencoded()</code> , <code>express.static()</code>
Third-party	Installed from NPM and loaded with <code>require()</code> .	<code>morgan (logging)</code> , <code>cors</code> , <code>cookie-parser</code>
Error-handling	Has four parameters (<code>err</code> , <code>req</code> , <code>res</code> , <code>next</code>). It handles errors passed by <code>next(err)</code> or thrown inside a route.	<code>app.use((err, req, res, next) => {...})</code>

5. Logging Middleware

A logging middleware records information about every incoming request, such as the time, the HTTP method, the URL, the status code, and the time taken to respond. Logs help in debugging, monitoring, and auditing an application. In this experiment a custom logger is written; the ready-made third-party package `morgan` performs the same task.

The logger stores the start time, registers a listener for the 'finish' event of the response (which is fired when the response has been sent), and calls `next()`. The status code is known only after the response is sent, so the log line is printed inside the listener.

6. Order of Middleware and Ending the Cycle

Middleware is executed strictly in the order of registration. The logger and the body-parsing middleware are registered before the routes so that they run for every request. The 404 handler and the error-handling middleware are registered after all the routes, because they must run only when no route has responded or when an error has occurred. A middleware such as `checkApiKey` can end the cycle early by sending a response without calling `next()`.

5. ALGORITHM / PROCEDURE

1. Start.
2. Create a project folder, run `npm init -y`, and install Express using `npm install express`.
3. Create `server.js`, load Express using `require('express')`, create the app, and set PORT to 3000.
4. Define `requestLogger(req, res, next)`: store the start time and the timestamp, register a `res.on('finish')` listener that prints the method, URL, status code, and

response time, and then call next().

5. Define checkApiKey(req, res, next): if the header x-api-key equals lab8-secret, call next(); otherwise respond with status 401.
6. Register app.use(requestLogger) first, and then app.use(express.json()).
7. Define the routes GET /, GET /about, POST /data, and GET /admin (with checkApiKey before its handler).
8. Add a 404 middleware and an error-handling middleware after all the routes.
9. Start the server using app.listen(PORT).
10. Run the server using node server.js.
11. Send requests from a browser, Postman, or curl, and observe the log lines printed in the terminal.
12. Stop.

6. PROGRAM

Terminal commands

```
npm init -y
npm install express
node server.js
```

server.js

```
// server.js - Express.js middleware demonstration (request logging)

// Step 1: Load Express and create the application
const express = require('express');
const app = express();
const PORT = 3000;

// Step 2: Custom logging middleware (application-level)
function requestLogger(req, res, next) {
  const start = Date.now();
  const time = new Date().toISOString();

  // Fires after the response is sent
  res.on('finish', () => {
    const ms = Date.now() - start;
    const line = `${req.method} ${req.originalUrl} -> ${res.statusCode}`;
    console.log(`[${time}] ${line} (${ms} ms)`);
  });

  // Hand over to the next middleware
  next();
}

// Step 3: Route-specific middleware (can end the cycle without calling next)
function checkApiKey(req, res, next) {
  if (req.headers['x-api-key'] === 'lab8-secret') {
    // Key is valid, continue to the route
    return next();
  }
  res.status(401).json({
    error: 'Unauthorized: valid x-api-key header required',
  });
}

// Step 4: Register the middleware (the order matters)
// Logger runs first for every request
app.use(requestLogger);
// Built-in middleware: parses the JSON body into req.body
app.use(express.json());

// Step 5: Define the routes
// Home page
app.get('/', (req, res) => {
  res.send('Welcome to the Express Middleware Demo');
});

// About page (JSON response)
app.get('/about', (req, res) => {
  res.json({ course: 'Full Stack Web Development', experiment: 8 });
});
```

```
});

// Receives JSON data and sends it back
app.post('/data', (req, res) => {
  res.json({ received: req.body });
});

// Protected route: the checkApiKey middleware runs before the handler
app.get('/admin', checkApiKey, (req, res) => {
  res.send('Welcome, admin! Access granted.');
```

});

// Step 6: 404 middleware (reached only when no route matched)

```
app.use((req, res) => {
  res.status(404).send('404 - Page Not Found');
});

// Step 7: Error-handling middleware (has four parameters)
app.use((err, req, res, next) => {
  console.error('Error:', err.message);
  res.status(err.status || 500).send('Something went wrong: ' + err.message);
});

// Step 8: Start the server
app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

7. CODE EXPLANATION

Code	Explanation
<code>const express = require('express');</code>	Loads Express, which is installed using <code>npm install express</code> .
<code>const app = express();</code>	Creates the Express application object.
<code>function requestLogger(req, res, next)</code>	Custom middleware function with the three parameters <code>req</code> , <code>res</code> , and <code>next</code> .
<code>const start = Date.now();</code>	Stores the start time so that the response time can be calculated later.
<code>res.on('finish', () => {...})</code>	Runs after the response has been sent. The status code is known at this point, so the log line is printed here.
<code>req.method, req.originalUrl, res.statusCode</code>	HTTP method, requested URL, and response status code shown in the log.
<code>next();</code>	Passes control to the next middleware or route. Without it, the request would hang.
<code>function checkApiKey(req, res, next)</code>	Route-specific middleware that protects a route.
<code>req.headers['x-api-key']</code>	Reads the <code>x-api-key</code> header of the request.
<code>res.status(401).json({...})</code>	Sends the error response and ends the cycle without calling <code>next()</code> .
<code>app.use(requestLogger);</code>	Registers the logger for all requests. It is registered first so that it sees every request.
<code>app.use(express.json());</code>	Built-in middleware that parses a JSON request body into <code>req.body</code> .
<code>app.get('/admin', checkApiKey, handler)</code>	Route with middleware. <code>checkApiKey</code> runs before the handler.
<code>res.json({ received: req.body })</code>	Returns the parsed body, which confirms that <code>express.json()</code> worked.
<code>app.use((req, res) => {...404...})</code>	Registered after the routes. It runs only when no route matched the request.
<code>app.use((err, req, res, next) => {...})</code>	Error-handling middleware, recognised by its four parameters.
<code>app.listen(PORT, ...)</code>	Starts the server and listens on port 3000.

8. TEST CASES AND EXECUTION DATA

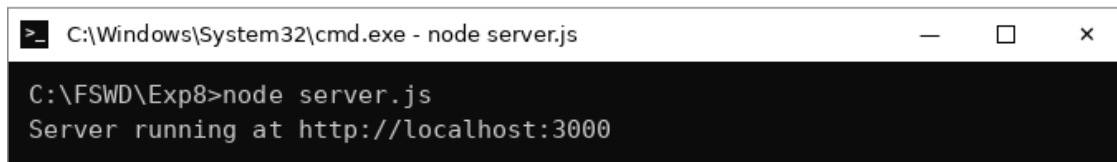
The following test cases were recorded with the server running in one Command Prompt and the requests sent using curl from a second Command Prompt.

Test Case TC-01

Test / Input	Open a Command Prompt in the project folder and run node server.js.
Expected Result	The terminal prints "Server running at http://localhost:3000" and the server keeps waiting for requests.
Actual Result	The server started and printed "Server running at http://localhost:3000". It kept running and waited for requests.
Status	Pass

TC-01 Output:

Command Prompt 1 - Server



```
> C:\Windows\System32\cmd.exe - node server.js

C:\FSWD\Exp8>node server.js
Server running at http://localhost:3000
```

Test Case TC-02

Test / Input	With the server running, open a second Command Prompt and run <code>curl http://localhost:3000/</code> and then <code>curl http://localhost:3000/about</code> .
Expected Result	The first request returns "Welcome to the Express Middleware Demo" and the second returns the JSON <code>{"course":"Full Stack Web Development","experiment":8}</code> . The server terminal prints one log line per request in the form <code>[timestamp] GET / -> 200 (x ms)</code> .
Actual Result	Both responses were received correctly. The server terminal printed the log lines <code>GET / -> 200</code> and <code>GET /about -> 200</code> with the time and the response time in ms.
Status	Pass

TC-02 Output:

Command Prompt 1 - Server

```
>_ C:\Windows\System32\cmd.exe - node server.js

C:\FSWD\Exp8>node server.js
Server running at http://localhost:3000
[2026-09-20T11:27:11.246Z] GET / -> 200 (3 ms)
[2026-09-20T11:27:11.256Z] GET /about -> 200 (1 ms)
```

Command Prompt 2 - Client

```
>_ C:\Windows\System32\cmd.exe

C:\FSWD\Exp8>curl http://localhost:3000/
Welcome to the Express Middleware Demo
C:\FSWD\Exp8>curl http://localhost:3000/about
{"course":"Full Stack Web Development","experiment":8}
C:\FSWD\Exp8>_
```

Test Case TC-03

Test / Input	Send a POST request to <code>http://localhost:3000/data</code> using curl with the header <code>Content-Type: application/json</code> and the body <code>{"name":"Sirisha","roll":"A24126552260"}</code> .
Expected Result	The response is <code>{"received":{"name":"Sirisha","roll":"A24126552260"}}</code> , which shows that <code>express.json()</code> parsed the body. The terminal logs <code>POST /data -> 200</code> .
Actual Result	The response <code>{"received":{...}}</code> was returned with the same name and roll number, which confirmed that <code>express.json()</code> parsed the body. The server logged <code>POST /data -> 200</code> .
Status	Pass

TC-03 Output:

Command Prompt 1 - Server

```
C:\Windows\System32\cmd.exe - node server.js

C:\FSWD\Exp8>node server.js
Server running at http://localhost:3000
[2026-09-20T11:27:11.246Z] GET / -> 200 (3 ms)
[2026-09-20T11:27:11.256Z] GET /about -> 200 (1 ms)
[2026-09-20T11:27:11.780Z] POST /data -> 200 (2 ms)
```

Command Prompt 2 - Client

```
C:\Windows\System32\cmd.exe

C:\FSWD\Exp8>curl -X POST http://localhost:3000/data -H "Content-Type: application/json" -d
{"name":"Sirisha","roll":"A24126552260"}
{"received":{"name":"Sirisha","roll":"A24126552260"}}
C:\FSWD\Exp8>_
```

Test Case TC-04

Test / Input	Run curl http://localhost:3000/admin first without any header and then again with the header x-api-key: lab8-secret.
Expected Result	Without the header the response is an error message and the log shows GET /admin -> 401 (checkApiKey ended the cycle). With the header the response is "Welcome, admin! Access granted." and the log shows GET /admin -> 200.
Actual Result	Without the header the 401 error message was returned and the log showed GET /admin -> 401, because the request stopped in checkApiKey. With the header the response was "Welcome, admin! Access granted." and the log showed GET /admin -> 200.
Status	Pass

TC-04 Output:

Command Prompt 1 - Server

```
>_ C:\Windows\System32\cmd.exe - node server.js

C:\FSWD\Exp8>node server.js
Server running at http://localhost:3000
[2026-09-20T11:27:11.246Z] GET / -> 200 (3 ms)
[2026-09-20T11:27:11.256Z] GET /about -> 200 (1 ms)
[2026-09-20T11:27:11.780Z] POST /data -> 200 (2 ms)
[2026-09-20T11:27:12.307Z] GET /admin -> 401 (6 ms)
[2026-09-20T11:27:12.313Z] GET /admin -> 200 (0 ms)
```

Command Prompt 2 - Client

```
>_ C:\Windows\System32\cmd.exe

C:\FSWD\Exp8>curl http://localhost:3000/admin
{"error":"Unauthorized: valid x-api-key header required"}
C:\FSWD\Exp8>curl http://localhost:3000/admin -H "x-api-key: lab8-secret"
Welcome, admin! Access granted.
C:\FSWD\Exp8>_
```

Test Case TC-05

Test / Input	Run curl http://localhost:3000/unknown.
Expected Result	The response is "404 - Page Not Found" (from the 404 middleware) and the terminal logs GET /unknown -> 404.
Actual Result	The response "404 - Page Not Found" was returned and the server logged GET /unknown -> 404.
Status	Pass

TC-05 Output:

Command Prompt 1 - Server

```
>_ C:\Windows\System32\cmd.exe - node server.js

C:\FSWD\Exp8>node server.js
Server running at http://localhost:3000
[2026-09-20T11:27:11.246Z] GET / -> 200 (3 ms)
[2026-09-20T11:27:11.256Z] GET /about -> 200 (1 ms)
[2026-09-20T11:27:11.780Z] POST /data -> 200 (2 ms)
[2026-09-20T11:27:12.307Z] GET /admin -> 401 (6 ms)
[2026-09-20T11:27:12.313Z] GET /admin -> 200 (0 ms)
[2026-09-20T11:27:12.841Z] GET /unknown -> 404 (0 ms)
```

Command Prompt 2 - Client

```
>_ C:\Windows\System32\cmd.exe

C:\FSWD\Exp8>curl http://localhost:3000/unknown
404 - Page Not Found
C:\FSWD\Exp8>_
```

9. OUTPUT

Output 1: Terminal log of all requests handled by the requestLogger middleware

Command Prompt 1 - Server (all requests logged)

```
C:\Windows\System32\cmd.exe - node server.js

C:\FSWD\Exp8>node server.js
Server running at http://localhost:3000
[2026-09-20T11:27:11.246Z] GET / -> 200 (3 ms)
[2026-09-20T11:27:11.256Z] GET /about -> 200 (1 ms)
[2026-09-20T11:27:11.780Z] POST /data -> 200 (2 ms)
[2026-09-20T11:27:12.307Z] GET /admin -> 401 (6 ms)
[2026-09-20T11:27:12.313Z] GET /admin -> 200 (0 ms)
[2026-09-20T11:27:12.841Z] GET /unknown -> 404 (0 ms)
```

Figure 1: Log lines printed by the logging middleware in the server terminal

Output 2: Requests sent from the client using curl

Command Prompt 2 - Client

```
C:\Windows\System32\cmd.exe

C:\FSWD\Exp8>curl http://localhost:3000/
Welcome to the Express Middleware Demo
C:\FSWD\Exp8>curl http://localhost:3000/about
{"course":"Full Stack Web Development","experiment":8}
C:\FSWD\Exp8>_
```

Figure 2: GET / and GET /about (TC-02)

Command Prompt 2 - Client

```
C:\Windows\System32\cmd.exe

C:\FSWD\Exp8>curl -X POST http://localhost:3000/data -H "Content-Type: application/json" -d
{"name":"Sirisha","roll":"A24126552260"}
{"received":{"name":"Sirisha","roll":"A24126552260"}}
C:\FSWD\Exp8>_
```

Figure 3: POST /data with a JSON body (TC-03)

Command Prompt 2 - Client

```
C:\Windows\System32\cmd.exe

C:\FSWD\Exp8>curl http://localhost:3000/admin
{"error":"Unauthorized: valid x-api-key header required"}
C:\FSWD\Exp8>curl http://localhost:3000/admin -H "x-api-key: lab8-secret"
Welcome, admin! Access granted.
C:\FSWD\Exp8>_
```

Figure 4: GET /admin without and with the x-api-key header (TC-04)

Command Prompt 2 - Client

```
C:\Windows\System32\cmd.exe

C:\FSWD\Exp8>curl http://localhost:3000/unknown
404 - Page Not Found
C:\FSWD\Exp8>_
```

Figure 5: GET /unknown handled by the 404 middleware (TC-05)

10. RESULT

Thus, the concept of **middleware** in Express.js and its working in the request-response cycle were studied. A custom logging middleware was developed that records the time, HTTP method, URL, status code, and response time of every incoming request, and the built-in `express.json()`, a route-specific middleware, a 404 middleware, and an error-handling middleware were demonstrated.